Contents

1 Vibevolts

Creating a new version of volts that runs faster using jules and other tools.

2 Done

2.1 Create new points instead of red satellties

- Do a little research to figure out how to create a nice 3d space filling set of points.
- create a nice grid of points to be observed for now.

2.2 exclusions Satellites and observatories

This test needs to be applied to any potential or actual pointing. It should be done across the whole vector.

- solar - is the sun within the FOV
- lunar - is the moon within the FOV
- earth - for satellite is the earth within FOV, for observatory make it above the horizon.

3 Notes

3.1 Numpy scatter gather

import numpy as np
    data = np.arange(18).reshape(6, 3) mask = np.array([True, False, True, False, True, False]) $\text{selected}_{\text{rows}}$ = data[mask] $\text{computed}_{\text{results}}$ = $\text{selected}_{\text{rows.sum}}$(axis=1, keepdims=True) $\text{final}_{\text{array}}$ = np.hstack([data, $\text{results}_{\text{container}}$])

4 Todo

4.1 visibility

- Create a 2-d array that has a flag for each time the satellite might be observed, a row for each time in the simulaton, and a vector of the times for the rows.
- for each target, compute if its within anyones FOV, if the observer isn't blinded, and the SNR is high enough.

4.2 Pointing selections (OK, this one will take some develoopement!)

4.3 Scoring

- use the visibilty matrix above
- take the visibility vector and calculate the fraction of the time that the thing is observed
- Calculate the gap times- basically for each column, compute the inter-observation distances and histogram them.

4.4 Details

4.4.1 Coding [5/13]

☒ At some point create a good builder function for the constellations

☒ Create the constellation

☒ create the appropirate targeting grid and rules for traversing it.

☒ Set up pytest

- ☒ first stage testing that displays things
- ☐ Get the timing in there as well: that is how long it takes to move forward a step
  - ☐ Add appropriate variables to the sensors
  - ☐ add appropriate function to radiometery
  - ☐ think about where you get the backgrounds
- ☐ write some testing code that shows where things are looking
- ☐ Create the targets with brightnesses.
- ☐ Build a function that will
  - ☐ iterate over all pairs
  - ☐ create a mask that includes only those things that are in field and not blinded
  - ☐ cut those out and process for limiting magnitude
  - ☐ fill back in an array of detected
- ☐ resolve issues with having multiple constellations, potentially with different names etc.
- ☐ write some code that computes what been detected and do a 3-D map
- ☐ iterate over some cycles and make sure things are evolving
- ☐ next build in something that stores these thing and computs gap times

4.4.2 Infrastructure [1/1]

- ☒ update your brew or whatever and get the google vetted gemini installed on Neptune.
- ☐ I could probably run a linter and a formatter across things

5 Prompts

5.0.1 LaTeX Documentation

Create a latex documentation file called vvdoc.tex. It should have the following sections: Create a table of content which will be based off the section structure you are generating below. Proceed as follows:

- read in an individual python file
- create a new section
- write a summary of the purpose of file
- for each function in the file, write a summary of the purpose of that funtion and the arguments and return value.
- be absolutely sure that for any underscores in the file, escape them with a backslash BEFORE you write them out to the latex file
- do this for each of files

Write another section summarizing what the demos are. Again be sure that the underscores in all text are preceeded by a backslash. Next, for each of the files, create a new section and output the contents of the file inside a new vebatim environment, but before you write a line wrap the text if it is longer than 100 characters. In side the verbatim environment you do not need to prefix a backslash to the underscore.

6 Architecture

6.1 Top-Level Keys

This document outlines the structure of the `sim_data` dictionary used in the VibeVolts simulation toolkit.

start$_{\text{time}}$ `datetime`

- **Description**: The starting time and date of the simulation. Must be a timezone-aware datetime object set to UTC.
- **Modified by**: `create_empty_simulation` (initialization)

delta$_{\text{time}}$ `float`

- **Description**: The time step for the simulation in seconds.
- **Modified by**: `create_empty_simulation` (initialization)

counts `dict`

- **Description**: A dictionary holding counts of various simulation objects.

- **Modified by**:
  - `create_empty_simulation` (initialization)
  - `add_celestial_bodies` (adds `celestial`)
  - `add_fixed_points` (adds `fixedpoints`)
  - `add_satellites_from_tle` (adds a satellite category)
  - `add_observatories` (adds `observatories`)
  - `geos` (modifies `satellites`)

pointing$_{\text{spheres}}$ `dict`

- **Description**: A dictionary to hold pre-computed pointing spheres. The keys are the number of points in the sphere.
- **Modified by**:
  - `create_empty_simulation` (initialization)
  - `generate_pointing_sphere` (adds a new pointing sphere)
  - `geos` (calls `generate_pointing_sphere`)

celestial `dict`

- **Description**: Holds data for celestial bodies (Sun and Moon).
- **Modified by**:
  - `add_celestial_bodies` (initialization)
  - `celestial_update` (updates positions)
- **Sub-keys**:

  position `np.ndarray` (2, 3) - Position vectors (x, y, z) in meters.
  velocity `np.ndarray` (2, 3) - Velocity vectors in m/s.
  acceleration `np.ndarray` (2, 3) - Acceleration vectors in m/s^2.

fixedpoints `dict`

- **Description**: Holds data for fixed reference points in the GCRS frame.
- **Modified by**:
  - `add_fixed_points` (initialization)
  - `update_visibility_table` (modifies `visibility`)
- **Sub-keys**:

  position `np.ndarray` (n, 3) - Position vectors of fixed points.

visibility `np.ndarray` (n, m) - Visibility table where rows are fixed points and columns are satellites. Value is 1 if visible, 0 otherwise.

satellites (and other satellite categories like `red_satellites`) `dict`

- **Description**: Holds data for a category of satellites.
- **Modified by**:
  - `add_satellites_from_tle` (initialization)
  - `geos` (appends to existing satellite data)
  - `propagate_satellites_new` (updates `position` and `pointing`)
  - `pointing_place_update` (updates `pointing_state`)
  - `jerk` (updates `pointing`)
  - `update_satellite_pointing` (updates `pointing`)
  - `update_visibility_table` (temporarily modifies `pointing`)
- **Sub-keys**:

  position `np.ndarray` (n, 3) - Position vectors in meters.

  velocity `np.ndarray` (n, 3) - Velocity vectors in m/s.

  acceleration `np.ndarray` (n, 3) - Acceleration vectors in m/s^2.

  orbital$_{\text{elements}}$ `np.ndarray` (n, 6) - Keplerian orbital elements.

  epochs `list[datetime]` - Epoch for each satellite's orbital elements.

  pointing `np.ndarray` (n, 3) - Pointing direction vector.

  pointing$_{\text{state}}$ `np.ndarray` (n, 2) - State of the pointing sequence for each satellite.

  detector `np.ndarray` (n, 9) - Detector properties for each satellite.

observatories `dict`

- **Description**: Holds data for ground-based observatories.
- **Modified by**: `add_observatories` (initialization), `propagate_observatories` (position and velocity updates)
- **Sub-keys**:

  position `np.ndarray` (n, 3) - Position vectors in meters.

  velocity `np.ndarray` (n, 3) - Velocity vectors in m/s.

  acceleration `np.ndarray` (n, 3) - Acceleration vectors in m/s^2.

pointing `np.ndarray` (n, 3) - Pointing direction vector.

latitude `np.ndarray` (n,) - Geodetic latitude in degrees.

longitude `np.ndarray` (n,) - Geodetic longitude in degrees.

altitude `np.ndarray` (n,) - Geodetic altitude in meters.

detector `np.ndarray` (n, 9) - Detector properties for each observatory.

Call $\text{create}_{\text{emptysimulation}}$ returns dictionary referred to as sim_{data} with $\text{start}_{\text{time}}$ $\text{delta}_{\text{time}}$ counts{} and pointing sheres{}

call $\text{add}_{\text{celesialbodies}}$ adds EMPTY celesial {} to sim_{data}

call $\text{add}_{\text{fixedpoitns}}$: adds argumnet fixedpoints'position' {} subarguments position np and visibuility np empty

7 Log

7.1 2025-10

7.1.1 *[2025-10-09 Thu]*

First figured out git rebase to clean up history a little, interactive. Pretty neat, but somthing I will need to review!

Lets go through files and try to organize what we have. Well, that may not have been too useful but I definately cleaned things up a little.

OK, let's look at and document how we build out a simulation. I realize I haven't really got this caged in my mind and it leads to inability to act, since I don't even know where to start.

1. prompt There are a large number of functions that add to or modify the dictionary sim_{data}. Please go through all the files and create a document that creates a list of all the dictionary elements added by anybody to sim_{data}, lists the types, and on the same lines lists what functions either add to or change that entry. If a function creates a dictionary or item inside another, document it the same way. Indent top level entries more if they are subentries. Put this data in a new file called datastructure.md, but do not change any other files.

2. Todo [3/5]

- ☐ Document the damn architecture in terms of how you build and run the simulation
- ☐ Check we are initializing the detector arrays and make sure you are filling all the fields.
- ☐ write a function in radiometry to compute it for a particular system
- ☐ Start building the functions to track what's seen - I probably need an outline for this.
- ☐ Check that vstack etc. are being used consistently when adding a new constellation. I think I'm doing this wrong.

7.1.2 *[2025-10-08 Wed]*

Update gemini-cli to 0.8.1. I thoght I did that yesterday, but I could have been mistaken.

Wow, I didn't write the physics and interpretation of dwell time down right AT ALL. Wriging it down from the book

$$t = \frac{\omega\beta\gamma^2}{\eta f^2 \alpha^2 A}$$

1. Todo [3/5]
   - ☒ Plot how bright something is as a function of distance. Choose a size, a band choose a solar angle, plot vs. distance out to lunar radii.
   - ☒ write down what I need get a search rate for a sensor
   - ☒ modify the sensor spec so it has those variables.
   - ☐ Check we are initializing the detector arrays and make sure you are filling all the fields.
   - ☐ write a function in radiometry to compute it for a particular system
   - ☐ Start building the functions to track what's seen - I probably need an outline for this.
   - ☐ Check that vstack etc. are being used consistently when adding a new constellation. I think I'm doing this wrong.

7.1.3 *[2025-10-07 Tue]*

Furloughed. I did make a little progress yesterday, well, not so much on this! Fixed less fairly quickly. Got stuck up in doing things imenu- actually that was pretty easy, start getting tangled up - should I say yak-shaving - on setting up LSP mode. i mean that's nice, but it's YAK SHAVING. STOP IT.

OK, now looking at creating a new simpler lambertian sphere function. OK, got that really simple plot going. Interesting hubrid work, I DID use gemini, but I did quite a bit by hand and used colab to solve a simple subproblem without a lot of ceremony.

1. Todo [5/9]

    - ☒ Get the damn vvdoc.tex file working in the background. I might give this task to jules.google.com if gemini is taken up with it.
    - ☒ Find and fix a bug in the less.tex document
    - ☒ Brew update underway, a new gemini-cli. Brew upgrade failed.
    - ☒ Install gemini-cli following following gemini instructions using NPM. Fast and smooth
    - ☒ Plot how bright something is as a function of distance. Choose a size, a band choose a solar angle, plot vs. distance out to lunar radii.
    - ☐ write down what I need get a search rate for a sensor
    - ☐ modify the sensor spec so it has those variables
    - ☐ write a function in radiometry to compute it for a particular system
    - ☐ Start building the functions to track what's seen - I probably need an outline for this.

7.1.4 *[2025-10-05 Sun]*

Took me a while to get going, but I wrote down some prioriities on the remarkable and they seem to be sticking.

OK, this is a little frustrating. Gemini is NOT able to get report.tex formatted right. I'm going to make sure everything is saved here and do a push to set jules loose on the problem in the background. After a WHOLE BUNCH of dicking around that worked. Quite frustrating. Took way to long.

1. Todo [1/5]

    - ☒ Get the damn vvdoc.tex file working in the background. I might give this task to jules.google.com if gemini is taken up with it.
    - ☐ write down what I need get a search rate for a sensor
    - ☐ modify the sensor spec so it has those variables
    - ☐ write a function in radiometry to compute it for a particular system
    - ☐ Start building the functions to track what's seen - I probably need an outline for this.

7.1.5 *[2025-10-04 Sat]*

Well this has been a lost week! I need to see where I'm at.

Well, I need to get things searching at the right rate. I think that depends on my adding a dwell time. Well, didn't get too much done, but I did get some minimal work on documentation. For whatever reason darn gemini is having a really hard time just fixing all the underscores to backslash underscore in report.tex that it's generating.

Also irritated by some emacs things, like not seeing files with spaces in them. checking out aquaemacs again.

7.2 2025–09

7.2.1 *[2025-09-28 Sun]*

It's been a week with not as much done as I wanted. Hmm, gemini says there's an update so I brew update, which is taking a while- looks like node wants to do a bunch of stuff. rename the old gemini-cli to gemini and updated node. That all took 16 minutes. Grrrr. Well, I hope I'm set up now.

Cogitation. I want the simplest thing that will work. So create a single grid for 20 deg FOV

Constellation.py needs to manage the creation of a constellation, which includes generating the satellites and their pointing characterisics.

Making some reasonable progress- well maybe not. I seem to be stuck on not getting the right argument to geos, which is confusing. Maybe I should start spyder.

- lunch

Dumped everything from the system over lunch now everything works now. Very odd. I wonder what was sticking? Add a new test that plots the RA and DEC vs time as we march through the space. That works. Somehow the plotting got messed up and wasted 20min making plots come out right.

Interesting. I think the word results or result was being overloaded somehow.

7.2.2 *[2025-09-21 Sun]*

OK logging in and fixing up a glitch in the GEMINI.md documentation "automatgically" after syncing things in. My set up was still going from yesterday.

Improving documentation, it seems like some type hints are being added. Good!

OK, what is next. I guess the next thing in my list above was creating a good builder function for the constellations. Spent a little time reading up on the propagation.py module, documenting it.

Vibe coded something to create a constellation, at least put the GEOs in place.

Later lets do some work on making a scan pattern thingy. Actually first, I think I want to make a global list of constellations to propogate. Actually let's save that for a little later.

Vibe code something to change the positions and a demo, but the pointing vector isnt getting updatd. I need to work on this next. I think it's time for a shower.

7.2.3 *[2025-09-20 Sat]*

It's been a bad couple of weeks in terms of making progress. I really need to review the code I have and then move forward. Working on Neptune, syncing stuff down to get things up to date.

- Remove my brew installed verion of gemini and put in the npm version. It wasn't updating right, just didn't trust it.
- OK, got fed up with trying to get gemini-cli reinstalled, got into yak shaving down to getting node reinstalled with brew which was bucking me. Bathroom and mess around.
- Watch some ER

- Reinstall node from nodes.org. Install gemini-cli. Things are working now, back to where I was!
- Later in the evening actually install and things all up on golden adorable. Try to get strong tex file up, but that didn't work. I need to try that again.

7.2.4 *[2025-09-15 Mon]*

No gemini-cli installed on GCE by default.

Well, I'm looking at things, but I'm not clear on the code, so I'm spending too much time thrashing.

7.2.5 *[2025-09-14 Sun]*

Long week without getting enough done. Looks like the first thing to try is the demo, which runs "in the office" (on GCE?) but not at home?

OK, now that seems to work. Who knows, maybe I had a corrupt python session. So what's next?

Maybe a distraction, but let's try to install google-cli. Homebrew.

OK, I think it's time to make the master structure an empty dictionary to start, and then have the various functions populate their pieces and add to the global. Ask gemini-cli to do this. This is the kind of refactoring that's irritating and problematic. I'ld like to see if gemini can do this!

7.2.6 *[2025-09-08 Mon]*

Try to run the same demo as yesterday in the office and here it runs fine. Confusing. I didn't take notes that were very good.

OK, trying to figure out what's useful to do here. i think I'll generate a nice LaTeX document I can print out to review tomorrow using Jules.

7.2.7 *[2025-09-07 Sun]*

Getting started in the afternoon. OK. Got distracted by tax reviews in the morning.

Looked up scatter gather above, I think this will make calculations a lot faster.

I think what I should do is work towards scoring some simple constellations, and get those done "by hand." So let's write an algorithm and generate some prompts. Write those above in details.

1. P1 Follow instructions in agents.org.

   ===== Create a new module called pointing vectors. In it create a new function called $\text{pointing}_{\text{vectors}}$(n) which creates n equally spaced points on the sphere using the fibonacci sphere algorhtm, and return the 3 dimensional points in an n by 3 numpy array. Also add a function to the module which draws the points on a 3-d sphere using plotly. Add a demo funciton to the demos that calls this function and displays it.

2. P2 Follow the instructions in agents.org. For each satellite, add two more parameters: first a "pointing count" which indicates what spherical pointing grid will be used. Second "pointing place" which indicates where in the sequence of pointing we are currently at. Create a function "pointing place update" that increments the pointing place value for all of the satellites, and if the pointing place is greater than or equal to pointing count resets pointing place to 0.

   Create another dictionary in the simulations state dictionary called pointing spheres. Entries will be indexed by the number of points the sphere is divided into, and the content will be numpy arrays created by the pointingVectors function.

   In the simulations state dicionary add another variable called $\text{delta}_{\text{time}}$ that refers to the time between time steps in the simulation.

7.2.8 *[2025-09-06 Sat]*

After reading an article and seeing how things were going, fed Jules the following prompt:

1. Prompt1

   Without changing any of the function signatures or functionality, refactor the python files so that there are ideally no more than 150 lines per file. Update the .org and .md documentation files so they are current. Before you proceed give me a plan telling me what functions go in what files.I don't know how this is going to work: but I'm going to try it out!

   Took 18 minutes. I need to review.

   Worked well! Spend some more time thinking about how to optimize search.

7.2.9 *[2025-09-04 Thu]*

Tried this one in Gemini but it didn't go well: will have to try again. One problem was that I was dragging things by hand, and even though there weren't many externals left out gemini created plugs to replace them. My bad. I need to be careful about context.

Consider the following two files. I would like to create a new testing function that creates a single satellite in a GEO location and initialized celestial bodies. Create a two diminsional grid with the first axis going from -pi/2 to pi/2 in 50 in increments correspnding to the latitude or declination, and the second axis going from 0 to 2 pi corresponding to longitude or right ascention in 100 steps. For each of these points, orient the pointing vector of the satellite in this direction and call the exclusion function. Place the return value in the array. Finally plot this array using plotly. Place this function in a new module separate from the one I am showing you.

7.2.10 *[2025-09-01 Mon]* Actuall did some good thinking and notes on

things to do. Started implementing thu.

7.2.11 *[2025-09-02 Tue]*

Made a little progress getting things working in the cloud on the work computer. Things needed to be moved.

7.2.12 *[2025-09-01 Mon]* Labor day.

I had left a thing running last night in jules, but it didn't finish right- it could be that things timed out so things didn't get synced. Anyway, asking it to fix the bug with graphics not being called right again. Prompt 2 from yesterday wasn't running.

I think I've been running too fast. Time to look at the code a little.

OK. Waste some time here. git mv rename this file. Figure out ^tab gets you between tabs in safari. Accidentally kill the Jules job I had fixing the error listed above. Figure out how to close a "window" in emacs. I guess this will all be useful but I wish I hadn't killed the job!

OK, it churns for some time- but it more or less turns out right this time at last. I need to edit some thigns.

Change which functions are called. Create a new funcTion just creates the $\text{demo}_{\text{plots}}$ file but isn't called by default (gemini).

Yeah, I think I really need to pay attention on how to get some results more interactively after each change to see that things are working - probably doing this in a jupyter notebook interactively makes sense. This is actually the kind of thing an AI should be able to do pretty well.

1. prompt 3
2. prompt 2 add a function $\text{dump}_{\text{detector}}$ that prints out a table with the rows representing all the detectors and the labeled columns being the different aspects of that detector.
3. prompt 1
    - Write function called jerk($\text{satellite}_{\text{number}}$) that takes takes the satellite indicated by that number and moves the pointing vector 0.3 radians in any direction.
    - write function that examines the $\text{exclusion}_{\text{table}}$, finds any satellite (column) which entirely 0, and applies the function jerk to that satellite.
    - create a new demo function that initializes the simulation, then creates and plots an $\text{exclusion}_{\text{table}}$, then applies the function just mentioned, and then creates and plots a new $\text{exclusion}_{\text{table}}$.

7.3 2025-08

7.3.1 *[2025-08-31 Sun]*

OK, lets get serious.

1. prompt 2 - ran overnight and didn't work
    - take all the demos that create plots in plotly, wrap them up in a new function that you run when you are testing things on the jules server and place the outputs in a new html file placed in the repo that can be viewed stand alone in the browswer.
    - neglected to tell it to leave the original demos there- got that in there on the prompt.
2. Prompt 1 - this is sort of a repeat worked after I changed it to not do so much testing. -read in and follow the instructions in agents.org file. -Modify the number of fixed points created is only 100 for the moment. -report any time you violate these instructions. -Create another array

visibility in overall structures that will have a number of columns equal to the number of satellites and a number of rows equal to the number of fixed points. It will be extended into a third dimension as the simulation proceeds. -change the function exclusion so that it returns a 0 if pointing is exclueded and 1 if it is not excluded. -check that the function $\text{create}_{\text{exclusiontable}}$ works with this new array and fills its elements by calling the fuction exclusion. -do not run all the demos, give me a change for GitHub.

7.3.2 *[2025-08-30 Sat]*

Well, that was a couple of weeks with not much going on. Got to stop that.

- An interesting idea that Hayden came up was that I might actually run some diagnostics in the virtual jules VM (or wherever, I guess!) and post them back to git. I kinda like that.
- i need to review status

Futzing around trying to get jules to autogenerate some good graphs of the code. Clearly this is some sort of yak shaving

7.3.3 *[2025-08-23 Sat]*

Hmm. Too bad I left some dead time here.

- Have vibevolts update all the documentation.

7.3.4 *[2025-08-18 Mon]*

Doing some coding in the hotel room in Kingman while Deborah gets ready to leave. Hmm. Well, that's kinda working, but somehow I am having some challenges getting git to the way I want it too. Ther are some edge cases i guess.

Later work a little when I get home, still maybe some problems.

OK, well, later, clone it on to neptune, which isn't too demanding intellectually, but a good thing to do if I'm going to work this in the long run. Establish a nice ssh key for push and pull in git and on the local machine and in the repo. Git copilot helped with that!

Well I think I'm getting the hang of it, but I really ought to write it down. For now, what's the next useful step I can take?

OK, I think I did something to do some visibility calculations. I haven't really RUN it though to check if things are working. Next.

7.3.5 *[2025-08-17 Sun]*

OK, I need to collect observations now. Let's get a prompt. Maybe see if jules can do this since it's across several files now.

7.3.6 Prompt

use the python tools currently in the repository, but don't change them unnecessarily. Create a new central data structure in vibevolts.py called fixedpoints that is initialized using the $\text{generate}_{\text{logsphericalpoints}}$ including points from 2000000 meters to tice geodistances. Add a new demo functino that plots this data in a plotly.

Did some reading on git- I thought it was all in my head, put creating local branches of remote things, switching branches, restoring older versions of files, and newer commans switch and resotre were not in my vocabulary.

7.3.7 *[2025-08-16 Sat]*

Last Socorro Vacation Day. Testing out working copy. Seems really good for some things! Took me 7 minutes to get my environment up.

OK, I need to take in to account points of view that are blocked by earth or blinded by the sun moon or earth. It would be nice to make this an ECS function- but let's start simple

1. Prompt - this appears to have mostly worked. a

   Based on the existing code you've just read, create a new python function exclusion in a new file that does the following.

   Add two global variables, $\text{earth}_{\text{radius}}$ and $\text{moon}_{\text{radius}}$ that contain those radii in meters. Create for me a function that takes an index number into the satellites array, and extracts position for the satellite, the pointing vector for the satellite, and also collects positions of the sun and the moon. Compute the unit vectors to the sun, the moon, and the earth from satellite position.

   For the sun, compute the angle betwen the vector to the sun and the pointing vector, and set a flag if the angle is less than the solar exclusion angle.

   For the moon and the earth, calculate the angle between the vector to the objects and the pointing angle, subtract the arctangent of the radius of the object and the distance to to the object, and set flags if either is less than the appropriate exclusion angle.

Set a global exclusion flag if any of these three flags is set and return this flag, either true of false.

For testing, create a function that that does some displays in plotly. The function should initialize the positions of the sun and moon. It should create a 100 satellites in random positions between leo out to geo each pointing in a random direction. Call the exclusion function. For each of these cases, using plotly, create a plot containing the earth, the satellites position with a pointing vector pointing away from it, and vectors to the moon, sun, and earth, together with an indication if the view was excluded or not.

7.3.8 *[2025-08-15 Fri]*

Summary: I actually did get a nice function to generate evenly spaced 3d points in, and get it tested. Working well with github.

Looking at the plan above, I wrote a prompt for gemini to create the space filling data. That worked, and I added a function to check it. There was a bug in that the radial distribution wasn't applired randomly in az and el, but gemini found that once I mentioned it. Checking in with git.

1. Prompt for Gemini I need an algorithm that will create a set of points in 3d space. Relative to a central point, they should be space logarithmically spaced in distance from the central point, but equally spaced in angle in any range of distances. Subject to these constraints the points should lie between an inner and an outer radius. Find this algorithm, and if possible give me code to execute it.

   take the function we just generated and add a new function that creates 4 plots: first, a 3d plot using plotly that displays the points (assuming we are in a Jupyter notebook), a plot that histograms the radii of the points, and plots that display the angular distributions of the points in terms of latitude and longitude. Display the function so I can copy it.

7.3.9 *[2025-08-14 Thu]*

Ok, lots of today has so far just been figuring out git and github and emacs and remembering those commands. I think I just need to download a nice git single page to put in my desk references.

OK, I'm seeing that I can actually do some editing on this in github itslef. It's OK I guess.

It's rather interesting to be moving these things around between github and other locations so quickly, and being able to edit thigns everywhere.

OK, the next action I need to do is to actually get radiometry working, and stuff like that.

1. Prompt1 Create a function called solarexclusion. Create an exclusion numpy vector. the same length as the number of satellites. Create a function which operates on all the satellites in the list of satellites in a vectorized manner. create a vector from the satellite to the sun and the vector representing the satellite pointing. If the angle between these two is less than the solar exclusion angle for the satellite, place a 1 in the exclusion list, othewise leave it as 0. Return this vector as well as a vector of the angle from the function.

   Create a test function that prints these two vectors out.